ripgrep `rg`

Operator Field Guide & Advanced Reference

A dense working reference for search design, pipelines, machine-readable output, PCRE2, multiline matching, scripting, and defensive auditing — for people already fluent in shells, regex, and large codebases.

Everything here was checked against a live install. Commands are copy-pasteable. macOS/POSIX assumed unless a line is tagged otherwise.

Verified environment

```
ripgrep 15.1.0
features: +pcre2
PCRE2 10.45 (JIT available)
platform: Darwin arm64 (macOS)
engine default: Rust regex (linear-time)
```

Flag names and short/long pairings in this guide were confirmed with `rg --help` on this exact version. Behavior may differ on much older builds.

NOTE

Scope of the security material. Every audit pattern in this guide finds *candidates*. Regex identifies strings that *look like* secrets, sinks, or persistence — it does not prove a credential is live, a call is exploitable, or intent is malicious. Triage every hit by hand.

Contents

1. Command Pattern Index **3**
2. Core Command Shapes **3**
3. Pattern Engineering **4**
3.1. Literal vs regex 4
3.2. Case control 4
3.3. Boundaries & anchors 5
3.4. Alternation · groups · repetition 4
3.5. Capture groups (for `-r` / `-o`) 4
3.6. Classes: bracket · POSIX · Unicode 4
3.7. Escaping: shell layer vs regex layer 5
3.8. Line-oriented model & performance traps 5
3.9. Pattern cards — recognition regexes 5
4. PCRE2 Engine **6**
4.1. Feature map 6
4.2. Worked examples 6
5. Multiline Search **7**
5.1. Worked examples (all are **candidates** — see §12) 7
6. File Selection & Ignore Behavior **8**
6.1. What is skipped by default 8
6.2. Override ladder 8
6.3. Examples 8
7. Globs & File Types **8**
7.1. Globs (`-g` / `--iglob`) 8
7.2. File types (`-t` / `-T`) 9
7.3. Custom types (`--type-add` , `--type-clear`) 9
8. Output Shaping **9**
8.1. Output profiles 10
9. Piping Into `rg` **10**
10. Piping `rg` Into Other Tools **10**
11. Git-Aware Workflows **11**
12. Security & Audit Recipes **11**
12.1. Credentials, keys, and tokens 11
12.2. Execution, download-exec, obfuscation 12
12.3. Persistence & platform config 12
12.4. Language-level dangerous sinks 12
12.5. Containers, Kubernetes, filesystem posture 12
13. Replacement & Extraction **12**
13.1. Extraction with `-o` (+ optional `-r`) 12
13.2. Migration previews (review before applying) 12
13.3. Safe migration workflow 13
14. JSON Mode & Machine-Readable Output **13**
14.1. Event stream 13
14.2. jq recipes (verified against jq 1.8.2) needs jq 13
15. Scripting With `rg` **14**
15.1. Exit codes (verified) 14
15.2. Quiet checks & guards 14
15.3. NUL-safe filenames & arrays 14
15.4. Pattern files, allow/deny lists 14
15.5. CI checks 14
16. Performance & Scale **15**
16.1. Levers, cheapest first 15
16.2. Profiles 15
17. Configuration **15**
18. Troubleshooting **16**
18.1. Symptom → cause → fix 16
18.2. Diagnosis commands 16
19. Expert Combos **16**
20. Appendix — Quick Reference **19**
20.1. A · Flags by purpose 19
20.2. B · Pattern & glob snippets 19
20.3. C · File types & ignore hierarchy 19
20.4. D · Output modes · exit codes · pipeline safety 19
20.5. E · PCRE2 & multiline quick reference 20
20.6. F · Common footguns (one-line) 20
20.7. G · Recommended command profiles 20

1. Command Pattern Index

One-screen lookup by intent. `PAT` = regex, `PATH` = optional path (defaults to `.`). Short flags are packed where legal (`-nP` , `-uu`). Verified against rg 15.1.0.

SEARCH & LOCATE	
Find matches	`rg -n --column 'PAT' PATH`
List matching files	`rg -l 'PAT'`
List non-matching files	`rg --files-without-match 'PAT'` — no short flag; `-L` is `--follow`
Count matching lines	`rg -c 'PAT'`
Count total matches	`rg --count-matches 'PAT'`
Extract match text only	`rg -o 'PAT'` · tighter: `rg -oP 'K(?=V)'`
Replace-preview (stdout)	`rg -r '$1' 'PAT'` — never edits files
Invert (lines without PAT)	`rg -v 'PAT'`
SCOPE & FILTER	
Search by file type	`rg -t py 'PAT'` · exclude: `rg -T js 'PAT'`
Glob include / exclude	`rg -g '*.rs' -g '!vendor/**' 'PAT'`
Override ignore rules	`rg -uu 'PAT'` (hidden+ignored) · `rg --no-ignore 'PAT'`
Search hidden, skip `.git`	`rg -n --hidden -g '!.git' 'PAT'`
Ignore only VCS rules	`rg -n --no-ignore-vcs 'PAT'`
Cap file size / depth	`rg --max-filesize 2M -d 6 'PAT'`
REPO & PIPELINE	
Tracked files only	`git ls-files -z | xargs -0 rg -n 'PAT'`
Changed vs base branch	`git diff --name-only -z main...HEAD | xargs -0 rg -n 'PAT'`
Staged (pre-commit)	`git diff --cached --name-only -z | xargs -0 rg -n 'PAT'`
Search command output	`CMD | rg -n 'PAT'` (stdin: no filename)
Search logs	`rg -N --no-heading 'ERROR|WARN|panic' /var/log`
ENGINES & MACHINE OUTPUT	
Multiline search	`rg -U '(?s)BEGIN.*?END'`
PCRE2 (lookaround, backref)	`rg -P '(?<=key=)\w+'`
Machine-readable	`rg --json 'PAT' | jq …`
NUL-delimited file list	`rg -0 -l 'PAT' | xargs -0 …`
Editor-ready lines	`rg --vimgrep 'PAT'`
SCRIPT · AUDIT · TUNE · DEBUG	
Safe scripting	`rg -q 'PAT' && …` · exit 0=hit, 1=none, 2=error
Security/audit sweep	`rg -nP -g '!*.min.js' 'SECRET_PAT'` (candidates only)
Performance profile	`rg -j 8 --max-filesize 4M -g '!vendor/**' 'PAT'`
Troubleshoot	`rg --debug 'PAT' 2>&1 | rg ignore` · `rg --files`

2. Core Command Shapes

Memorize these fifteen. Each is a spine you decorate with globs, types, and context flags. `shines` = where it earns its place; `foot` = the trap.

`rg -n --column 'PAT' PATH`

Line + column for every match.
shines editor/tool jumps; base of `--vimgrep`.
foot `--column` is a 1-based **byte** offset, not a codepoint — off for multibyte lines.

`rg --files-without-match 'PAT'`

Files that never match — coverage gaps.
shines "which files lack a license header / test import".
foot **no short flag.** `-L` means `--follow`, not this.

`rg --count-matches 'PAT'`

Count every match, including repeats per line.
shines true occurrence totals for reports.
foot with `-o` semantics differ subtly on overlap; regex can't overlap-match.

`rg -r 'REPL' 'PAT' PATH`

Print matches rewritten — preview only.
shines rename/migration diffs; `$1` capture refs.
foot **does not touch files.** stdout only. Pipe to `sd` / `perl` to apply.

`rg --files PATH`

List every file rg **would** search — no pattern.
shines verifying ignore/type filters; candidate lists.
foot honors ignores/hidden exactly like a real search; that's the point.

`rg -0 -l 'PAT' | xargs -0 CMD`

NUL-safe path handoff to another tool.
shines any batch op over matched files, safely.
foot both sides must speak NUL: `-0` here **and** `xargs -0`.

`rg -n --no-ignore-vcs 'PAT'`

Ignore `.gitignore`, keep `.ignore` / `.rgignore`.
shines auditing gitignored build output deliberately.
foot not the same as `--no-ignore` (which drops **all** ignore files).

`rg -n -U 'MULTILINE_PAT'`

Let a single match span newlines.
shines key blocks, stanzas, `RUN …` line-continuations.
foot `.` still excludes `\n` unless `(?s)` / `--multiline-dotall`; can buffer big files.

`rg -l 'PAT' PATH`

Print only paths that contain a match.
shines feeding `xargs`; scoping a second pass.
foot newline-delimited → breaks on odd names. Use `-0` for pipelines.

`rg -c 'PAT' PATH`

Count of matching **lines** per file.
shines hot-file ranking; quick density check.
foot one line with 5 hits counts as 1. Use `--count-matches` for totals.

`rg -o 'PAT' PATH`

Emit only the matched substring.
shines extraction (URLs, hashes, keys) → `sort -u`.
foot each match on its own line; capture groups are **not** isolated (use `-r`).

`rg --json 'PAT' PATH`

Structured JSONL event stream.
shines robust parsing; audit artifacts; TSV via `jq`.
foot bytes are base64 when not valid UTF-8 (`text` vs `bytes` key).

`rg --files -g 'GLOB'`

File list constrained to a glob.
shines "what will `-g '*.tf'` actually hit?"
foot quote the glob or the shell expands it first (esp. zsh).

`rg -n --hidden -g '!.git' 'PAT'`

Include dotfiles, still skip the `.git` DB.
shines dotfile/config audits without object-store noise.
foot `--hidden` alone re-adds `.git/**`; keep the exclude glob.

`rg -n -P 'PCRE2_PAT'`

Switch to PCRE2 for lookaround / backrefs.
shines `(?<=…)`, `(?=…)`, `\1`, atomic groups.
foot loses linear-time guarantee; can catastrophically backtrack.

3. Pattern Engineering

The default engine is Rust `regex`: linear-time, no lookaround, no backreferences. That constraint is a feature — it can't catastrophically backtrack. Reach for `-P` (§4) only when you truly need what it lacks.

3.1. Literal vs regex

`-F` / `--fixed-strings` disables regex — the whole pattern is literal. Best for searching code that is **itself** regex/glob, version strings, or paths.

```
rg -F 'a.b[c]*'        # finds the literal text a.b[c]*
rg    'a.b[c]*'        # regex: a, any char, b, zero+ c
rg -Fw 'id'            # literal + word boundary
```

3.2. Case control

Default is **case-sensitive**. Opt into the others explicitly:

```
rg -i 'error'          # ignore case always
rg -S 'Error'          # smart-case: insensitive iff all-lowercase
rg -s 'Error'          # force case-sensitive (override config -i)
```

3.4. Alternation · groups · repetition

```
rg 'cat|dog'                # alternation
rg '(?:ba)?sh\b'            # non-capturing group
rg 'a{2,4}'                 # counted repetition
rg 'colou?r'                # optional
```

3.5. Capture groups (for `-r` / `-o`)

```
rg -o -r '$1' '"(\w+)":'     # pull JSON-ish keys
rg -r '${1}_new' '(\w+)_old'  # brace form when digit-adjacent
```

3.6. Classes: bracket · POSIX · Unicode

```
rg '[A-Fa-f0-9]{7,40}'        # bracket expression
rg '[[:xdigit:]]{7,40}'       # POSIX class (same idea)
rg '\p{Lu}\p{Ll}+'            # Unicode: Upper then lowers
```

3.3. Boundaries & anchors

```
rg -w 'id'              # \bid\b, but faster to type
rg    '\bTODO\b'
rg    '^\s*import '     # ^ = start of LINE (line-oriented)
rg    ';\s*$'           # $ = end of LINE
```

⚠ **FOOTGUN**
`^` and `$` anchor to **line** edges, not file edges — rg is line-oriented by default. Under `-U` they still mean line edges; use `\A` / `\z` (PCRE2) for buffer edges.

```
rg '\w+@\w+'                    # \w is Unicode-aware by default
```

✓ **FIELD TIP**
POSIX classes (`[[:alpha:]]` , `[[:space:]]` , `[[:xdigit:]]`) are portable across the default engine and PCRE2 and read cleanly in audit patterns.

3.7. Escaping: shell layer vs regex layer

Two separate escape problems. The shell sees the string first; the regex engine sees whatever survives. Single-quote the pattern so the shell keeps its hands off, then escape **regex** metacharacters with backslashes.

You want to match	Pattern (single-quoted)
a literal dot	`rg '\.'` or `rg -F '.'`
a literal `$`	`rg '\$'` (regex) — or `rg -F '$'`
a literal `(`	`rg '\('`
a path `a/b.c`	`rg -F 'a/b.c'` (simplest)
a dollar-var **not** expanded	`rg 'PATH=\$HOME'` in single quotes
a single quote inside pattern	`rg 'it'\''s'` (close-quote, escaped `'` , reopen)

⚠ **FOOTGUN**
Double quotes let the shell expand `$var` , backticks, and `!` (bash history). A pattern like `"$1"` becomes empty. Default to **single** quotes for patterns; switch to double only when you deliberately want interpolation.

3.8. Line-oriented model & performance traps

rg scans line by line. A pattern can't match across `\n` without `-U` . Within a line, catastrophic constructs are cheap in the default engine (no backtracking) but murderous under `-P` .

Trap	Why / fix
`.*.*` / nested quantifiers	Fine in default engine; can blow up in PCRE2. Prefer specific classes.
Unbounded `.*` for extraction	Greedy; grabs to line end. Use `[^…]*` or lazy `.*?` (PCRE2/ `-U`).
Leading `.*`	Usually pointless — rg already searches anywhere in the line.
Huge alternations of literals	Use `-f patterns.txt` (one per line); rg builds one Aho-Corasick automaton.
Unicode when you meant bytes	`\w` , `\d` are Unicode. For ASCII-only speed use `(?-u:\w)` or `[[:word:]]` .

3.9. Pattern cards — recognition regexes

Copy-pasteable **candidate** detectors. All use the default engine unless tagged `needs PCRE2` . They flag strings that *look like* the artifact; they prove nothing on their own.

IPv4 candidate

```
\b(?:\d{1,3}\.){3}\d{1,3}\b
```

Matches 999.999.999.999 too. Range-check downstream.

IPv6 rough

```
\b(?:[A-Fa-f0-9]{1,4}:){2,7}[A-Fa-f0-9]{1,4}\b
```

Skips `::` compression forms. Rough triage only.

Email candidate

```
\b[[:alnum:]._%+-]+@[[:alnum:].-]+\.[[:alpha:]]{2,}\b
```

Over/under-matches per RFC 5322. Good enough to grep.

URL candidate

```
\bhttps?://[^\s"'<>)]+
```

Stops at whitespace/quote/bracket. Trailing `.` may cling.

Domain candidate

```
\b(?:[[:alnum:]-]+\.)+[[:alpha:]]{2,}\b
```

Also hits `foo.bar` in prose. Noisy without context.

UUID

```
\b[0-9a-fA-F]{8}-(?:[0-9a-fA-F]{4}-){3}[0-9a-fA-F]{12}\b
```

Any version/variant. Add anchors for strict.

SHA-256

```
\b[0-9a-f]{64}\b
```

Also matches any 64 hex chars. Context matters.

SHA-1 / MD5 (legacy)

```
\b[0-9a-f]{40}\b   \b[0-9a-f]{32}\b
```

40=SHA-1, 32=MD5. Legacy-hash indicator only.

JWT-looking token

```
\beyJ[A-Za-z0-9_-]{8,}\.[A-Za-z0-9_-]{8,}\.[A-Za-z0-9_-]{8,}
```

`eyJ` = base64 `{"` . Three dot-parts.

PEM private key header

```
-----BEGIN (?:RSA |EC |OPENSSH |DSA |PGP )?PRIVATE KEY-----
```

Header only; see §5 for full block via `-U`.

AWS access key id

```
\b(?:AKIA|ASIA|AGPA|AIDA)[0-9A-Z]{16}\b
```

Key **id**, not secret. Presence ≠ leak.

Generic secret assign

```
(?i)(?:pass(?:wd|word)?|secret|api[_-]?key|token)\s*[:=]\s*['"][^'"]{6,}['"]
```

Huge false-positive rate. Pair with §12.

TODO / FIXME / HACK

```
\b(?:TODO|FIXME|HACK|XXX|BUG)\b
```

Add `:` or `\(` to cut prose noise.

Shell download-exec

```
\b(?:curl|wget)\b[^|]*\|\s*(?:sudo\s+)?(?:ba)?sh\b
```

Classic `curl … | sh`. Also legitimate installers.

base64 decode call

```
\bbase64\s+(?:-d|-D|--decode)\b
```

macOS uses `-D`, GNU uses `-d`. Both here.

eval / exec call

```
\b(?:eval|exec)\s*\(
```

Language-agnostic sink. Comments included.

SQL string concat

```
(?i)\b(?:select|insert|update|delete)\b.*(?:\+\s*['"]|['"]\s*\+|%s|\$\{)
```

Injection **candidate**, not proof.

Python subprocess

```
subprocess\.(?:run|call|Popen|check_output)\([^)]*shell\s*=\s*True
```

`shell=True` + dynamic arg = review.

Node child_process

```
\b(?:exec|execSync|spawn)\s*\(|require\(['"]child_process['"]\)
```

Import or call site. Triage inputs.

> ⚠ **DANGER**
> **These are candidate detectors, not verdicts.** A regex hit means “a human should look,” never “this is a vulnerability” or “this is malicious.” False positives are the norm in real repos. Do not paste these into a pipeline that auto-blocks or auto-reports without manual triage.

4. PCRE2 Engine

`-P` / `--pcre2` swaps the linear-time Rust engine for PCRE2 10.45 (JIT on this build). You gain lookaround, backreferences, atomic/possessive groups, named groups, and richer inline flags. You lose the linear-time guarantee — a bad pattern can backtrack catastrophically. Use it for what the default engine **cannot** express, not by default.

4.1. Feature map

Construct	Syntax / example
Lookahead (pos / neg)	`foo(?=bar)` · `foo(?!bar)`
Lookbehind (pos / neg)	`(?<=bar)foo` · `(?<!bar)foo` — fixed / bounded width
Reset match start	`prefix\K rest` — drops `prefix` from the reported match
Non-capturing group	`(?:…)` (also in default engine)
Lazy quantifier	`.*?` `.+?` `\w{2,}?` (also under `-U` default engine)
Named group	`(?<key>\w+)` → backref `\k<key>`, replace `$key`
Backreference	`(\w+)\s+\1` — repeated token
Atomic / possessive	`(?>…)` · `a++` `a*+` — prevent backtracking
Inline flags (scoped)	`(?i)` global · `(?i:foo)` scoped · `(?s)` dotall · `(?m)` multi
Buffer anchors	`\A` start-of-input · `\z` end-of-input (vs line-based `^` / `$`)

4.2. Worked examples

```
# Extract a secret value but drop everything left of it (\K), leave quotes off
rg -oP 'api_key\s*=\s*"\K[^"]+'

# password= assignment NOT followed by an allowed placeholder (neg. lookahead)
rg -P 'password\s*=\s*"(?!(?:REDACTED|CHANGEME|EXAMPLE|\${|<))[^"]{6,}"'

# risky eval( on lines that are (approximately) not comments
rg -P '^(?!\s*(?://|#|\*)).*\beval\s*\('

# extract Python import targets (\K keeps only the module path)
rg -oP '^\s*(?:from|import)\s+\K[\w.]+'

# extract JS/TS import source between quotes
rg -oP "(?:from|require\()\s*['\"]\K[^'\"]+"
```

```
# tighter URL: stop before trailing sentence punctuation (neg. lookbehind)
rg -oP 'https?://[^\s"''<>]+(?<![.,;:)\]])'

# suspicious one-liner: download piped to a shell within ~40 chars
rg -P '\b(?:curl|wget)\b.{0,40}\|\s*(?:sudo\s+)?(?:ba)?sh\b'

# duplicate-ish self-assignment  x = x  (backreference)
rg -P '\b(\w+)\s*=\s*\1\b'

# lookbehind for a config key's value (fixed-width key)
rg -oP '(?<=^AllowUsers )\S.*'

# lookahead for basename before an image extension
rg -oP '\w[\w-]*(?=\.(?:png|jpe?g|svg|gif)\b)'
```

⚠ **FOOTGUN**
PCRE2 lookbehind must be **fixed-width** (or a bounded alternation of fixed-width branches). `(?<=user(name)?=)` fails. Sidestep with `\K`, which marks "report from here" and has no width restriction.

⚠ **FOOTGUN**
Catastrophic backtracking is real under `-P`. Patterns like `(a+)+$` or `(.*,)*` against adversarial input can hang. The default engine is immune. Keep PCRE2 patterns anchored and specific; prefer possessive quantifiers (`.*+`) or atomic groups `(?>…)` when nesting is unavoidable.

NOTE
Portability. `-P` requires ripgrep built with `+pcre2` (this build has it; check `rg --version` or `rg --pcre2-version`). Patterns using `\K`, lookbehind, or possessive quantifiers will not run on the default engine or on `grep` / `sed`. Keep an engine-agnostic fallback for CI that may run elsewhere.

When not **to use PCRE2:** simple literals, alternations, classes, counted repetition, and word/line anchors are all faster and safer in the default engine. Only cross over for lookaround, backreferences, or `\K`.

5. Multiline Search

By default rg is line-oriented: a match cannot cross `\n`. `-U` / `--multiline` lets a pattern span line boundaries. Two independent switches matter:

Flag / token	Effect
`-U`, `--multiline`	Allow the **pattern** to match across newlines. `.` still excludes `\n`.
`--multiline-dotall`	Make `.` match `\n` too (global DOTALL). Implies `-U`.
`(?s)` inline	Scoped DOTALL — same as above but local to the pattern.
`(?m)` inline	`^` / `$` match at every internal line edge within the buffer.
`\A` / `\z`	Anchor to buffer start/end (PCRE2) rather than line edges.

Under `-U`, rg matches against a larger buffer, so `.*` can gobble to end of file. Prefer lazy `.*?` and negated classes `[^…]`. Multiline disables some line fast-paths and holds more in memory; on multi-GB files it costs both time and RAM.

5.1. Worked examples (all are candidates — see §12)

```
# PEM private key block, lazily bounded to its own END line
rg -U '(?s)-----BEGIN [A-Z ]*PRIVATE KEY-----.*?-----END [A-Z ]*PRIVATE KEY-----'

# YAML secret block: a `secrets:` key followed by indented children
rg -U '(?m)^\s*secrets:\n(?:[ \t]+\S.*\n?)+'

# HTML <script> block (dotall so . spans lines)
rg -U --multiline-dotall '<script\b[^>]*>.*?</script>'

# JS import of child_process followed by an exec( sink later in the file
rg -U '(?s)\bchild_process\b.*?\bexec(?:Sync)?\s*\('

# Python function block, approximate: def … up to next top-level line (needs -P: lookahead)
rg -UP '(?s)^def \w+\([^)]*\):.*?(?=\n\S)'

# multi-line TODO: a TODO comment continued on following comment lines (needs -P)
rg -UP '(?s)#\s*TODO:.*?(?=\n\s*[^#\s]|\n\s*\n)'

# Dockerfile RUN with backslash line-continuations
rg -U 'RUN(?:[^\n]*\\\n)+[^\n]*'

# INI/TOML-ish stanza: a [section] up to the next section or EOF (needs -P: lookahead + \z)
rg -UP '(?s)\[database\].*?(?=\n\[|\z)'

# JSON-ish object containing a password key  (prefer jq — see note)
rg -U '(?s)\{[^{}]*"password"[^{}]*\}'
```

⚠ **FOOTGUN**
`-U` enables **multiline**, not **PCRE2**. Lookaround (`(?=…)` , `(?<=…)`) and buffer anchors (`\z`) still require `-P` . Combine them as `-UP` — several block-boundary patterns above depend on it. Without `-P` rg errors out with "look-around … is not supported."

⚠ **FOOTGUN**
Matching structured formats (JSON, YAML, HTML, real code) with regex is **approximate** — nesting, quoting, and comments break it. Use `-U` for a fast first-pass triage, then reach for a real parser: `jq` for JSON, `yq` for YAML, a DOM/AST tool for HTML/code. Never gate a build on a regex "parse" of JSON.

✓ **FIELD TIP**
`-U` beats piping to `perl -0777 -ne` when you want to keep rg's gitignore, file-type, glob, hidden-file, and colorized-output machinery. Drop to `perl` only for transforms rg can't express, or when you need true whole-file slurp semantics with capture-heavy substitution.

6. File Selection & Ignore Behavior

Recursion from `.` (or given PATHs) is the default. rg silently applies a stack of filters; most "why didn't it match?" incidents (§18) are a filter, not the regex.

6.1. What is skipped by default

Filter	Source / meaning
VCS ignores	`.gitignore` (per-dir + parents), `.git/info/exclude` , global `core.excludesFile`
Non-VCS ignores	`.ignore` , `.rgignore` (rg/ag/ack convention; not tied to git)
Hidden files	Anything beginning with `.` — includes `.git/` , `.env` , dotfiles
Binary files	Detected by NUL byte; content-searched only with `-a` / `--text` or `-uuu`
Symlinks	Not followed during traversal unless `-L` / `--follow`

6.2. Override ladder

Flag	Turns off
`--no-ignore-vcs`	`.gitignore` + git excludes only (keeps `.ignore` / `.rgignore`)
`--no-ignore-dot`	`.ignore` / `.rgignore` (dot-ignore) files
`--no-ignore-parent`	ignore files in **parent** directories above the search root
`--no-ignore`	ALL of the above (dot, exclude, global, parent, vcs)
`--hidden`	the hidden-file skip (dotfiles become searchable)
`--ignore-file FILE`	**adds** an extra ignore file (gitignore syntax)
`-u` / `-uu` / `-uuu`	`--no-ignore` / `+ --hidden` / `+ --text` (binary as text)

⚠ **FOOTGUN**
`-uu` (and `--hidden` without an exclude) re-includes `.git/**` — thousands of object files, hooks, and packs. Verified: `rg -uu -l PAT` in a repo surfaces `.git/hooks/*` . For hidden-file audits always add `-g '!.git'` (or `-g '!.git/**'`).

6.3. Examples

```
rg -n --hidden -g '!.git' 'PATTERN'          # dotfiles, minus the git store
rg -n --no-ignore -g '!.git' -g '!node_modules' 'PATTERN'   # ignored files, sane excludes
rg -n --one-file-system 'PATTERN' /          # don't cross mount points
rg -n --max-filesize 5M 'PATTERN'            # skip anything larger than 5 MiB
rg -n -L 'PATTERN' src/                       # follow symlinks (watch for loops)
rg -n --ignore-file .auditignore 'PATTERN'   # project-specific extra excludes
rg --files --no-ignore --hidden -g '!.git' | rg -i 'secret|\.pem$|id_rsa'  # audit what's ignored
```

NOTE
`--search-zip` / `-z` is present in this build. It transparently searches inside `*.gz` , `*.bz2` , `*.xz` , `*.lz4` , `*.zst` , etc., by shelling out to the matching decompressor — so it depends on `gzip` / `xz` / `zstd` being installed. For arbitrary preprocessing use `--pre CMD` (e.g. `pdftotext` , `strings`); both add per-file process overhead. needs external decompressors

7. Globs & File Types

7.1. Globs (`-g` / `--iglob`)

`-g GLOB` filters by gitignore-style glob; `!` negates; later globs win; `**` spans directories. `--iglob` is the case-insensitive form.

```
rg -g '*.rs' 'PATTERN'                    # include only Rust
rg -g '!*_test.go' 'PATTERN'              # exclude test files
rg -g '*.{ts,tsx}' 'PATTERN'              # brace expansion (rg-side, quote it!)
rg -g 'src/**/*.py' 'PATTERN'             # path-scoped recursive
```

```
rg --iglob '*.MD' 'PATTERN'                # case-insensitive glob
rg -g '!{node_modules,dist,build,target,vendor,.venv,__pycache__,coverage}/**' 'PATTERN'
```

⚠ **FOOTGUN**
Quote every glob. Unquoted `*.rs` is expanded by the shell **before** rg sees it — in zsh an unmatched glob is a hard error (`no matches found`); in bash it may pass through literally or expand to files in the cwd only. Single quotes hand the raw glob to rg.

7.2. File types (`-t` / `-T`)

`-t NAME` restricts to a built-in or custom type; `-T NAME` excludes it; `--type-list` prints all definitions. These are verified present on rg 15.1.0.

Goal	Command
Only shell	`rg -t sh 'PATTERN'`
Only Python	`rg -t py 'PATTERN'`
Only JS **and** TS	`rg -t js -t ts 'PATTERN'`
Only Markdown	`rg -t md 'PATTERN'`
Only config-ish	`rg -t config -t toml -t yaml 'PATTERN'`
Dockerfiles	`rg -t docker 'PATTERN'` (glob `*Dockerfile*`)
Terraform / HCL	`rg -t tf 'PATTERN'`
Only logs	`rg -t log 'PATTERN'`
Everything **but** tests	`rg -T test 'PATTERN'` (if defined) or `-g '!*_test.*'`
Exclude minified JS	`rg -t js -g '!*.min.js' 'PATTERN'`

⚠ **FOOTGUN**
There is **no** built-in `k8s` / `kubernetes` type on rg 15.1.0. For Kubernetes manifests use `-t yaml` plus a content filter, or a custom type (below). Don't assume a type exists — confirm with `rg --type-list | rg NAME`.

7.3. Custom types (`--type-add`, `--type-clear`)

Define a type inline for one run, or persist it in `.ripgreprc` (§17). Syntax is `name:glob` (repeat `--type-add` for multiple globs), then use it with `-t`.

```
# dotfiles as a type
rg --type-add 'dot:.*' -t dot 'PATTERN'

# IaC bundle: terraform + k8s-ish yaml + helm
rg --type-add 'iac:*.tf' --type-add 'iac:*.tfvars' \
   --type-add 'iac:*.yaml' --type-add 'iac:Chart.yaml' -t iac 'PATTERN'

# logs including rotated files
rg --type-add 'logs:*.log' --type-add 'logs:*.log.*' -t logs 'ERROR|panic'

# redefine an existing type from scratch
rg --type-clear js --type-add 'js:*.mjs' -t js 'PATTERN'
```

✓ **FIELD TIP**
Persist project types in `.ripgreprc` and point `RIPGREP_CONFIG_PATH` at it (§17), so `-t iac` / `-t logs` work everywhere without the long flag soup.

8. Output Shaping

Flag	Effect
`-n` / `-N`	Show / hide line numbers (`-n` is default on a TTY, off in pipes)
`--column`	1-based **byte** column of the first match on the line
`--heading` / `--no-heading`	Group matches under a filename header / one line per match
`-H` / `-I`	Force / suppress the filename prefix (`--with-filename` / `--no-filename`)
`-o`	Print only the matched substring
`-r REPL`	Rewrite matches on output (`$1`, `${name}`); stdout only
`--passthru`	Print **every** line; matches still highlighted — grep-as-filter off
`-A/-B/-C N`	After / before / around context lines
`--trim`	Strip leading whitespace from output lines
`--max-columns N`	Suppress lines longer than N (minified/binary guard)
`--max-columns-preview`	…but still show a truncated preview of them
`--color=WHEN`	`auto` / `always` / `never` — use `always` before a pager
`--colors SPEC`	Restyle, e.g. `--colors 'match:fg:red' --colors 'path:style:bold'`
`--vimgrep`	`file:line:col:text`, one match per line — editor quickfix format

`--json`	JSON Lines event stream (§14)
`-0` / `--null`	NUL-terminate **filenames** (with `-l` / `--files`) for safe `xargs -0`
`--null-data`	Treat **input** as NUL-separated records instead of lines
`--field-match-separator`	Change the `:` between path/line/col fields
`--field-context-separator`	Change the `-` separator on context lines
`--path-separator`	Force forward-slash vs back-slash in printed paths (cross-platform output)
`--sort / --sortr KEY`	Deterministic order by `path` / `modified` /… — **single-threaded**

8.1. Output profiles

Human (default TTY / -p)

Heading, color, line numbers, grouped.

```
rg -p 'PATTERN'        # force pretty even in a pipe
```

Editor / quickfix

One match per line, jumpable.

```
rg --vimgrep 'PATTERN'
```

Script-friendly

Stable, no color, no heading.

```
rg -N --no-heading --color=never 'PATTERN'
```

Null-safe file list

For batch tools; survives odd names.

```
rg -0 -l 'PATTERN' | xargs -0 CMD
```

JSON (machine)

Robust parsing; see §14.

```
rg --json 'PATTERN' | jq -r '…'
```

Report-friendly

Sorted, trimmed, minified-safe.

```
rg -n --heading --sort path --trim \
   --max-columns 200 --max-columns-preview
'PATTERN'
```

⚠ FOOTGUN
`--color=always` embeds ANSI escapes. Great into `less -R`, poison into a file or a parser — you get `\e[...m` litter. Use `--color=never` (or rely on `auto`) whenever the sink is not a terminal. For machine output prefer `--json`, never colorized text.

9. Piping Into `rg`

rg reads stdin when it has no path argument and stdin is not a TTY (or when you pass `-`). **Stdin search has no filename** — every match line is bare text, so you lose the path/line context that direct file arguments give you.

Form	Consequence
`rg PAT file`	Filename + line number kept; rg can mmap and parallelize
`cat file | rg PAT`	One anonymous stream; no path, no per-file parallelism — an anti-pattern
`CMD | rg PAT`	Correct use of stdin: the **only** source is the command's output

```
ps aux | rg -i 'ssh|vpn|proxy'                    # scan process table
brew list --formula | rg 'ssl|crypto|python'       # macOS; needs: brew
git log --oneline --decorate --all | rg 'fix|security|auth|token'   # needs: git
docker ps -a | rg 'healthy|unhealthy|exited'       # needs: docker

# Generate candidate files with find/fd, hand off via NUL — never word-split
find . -type f -name '*.log' -print0 | xargs -0 rg -n 'ERROR|WARN|panic'   # POSIX
fd -0 -e js -e ts | xargs -0 rg -n 'eval|Function\('  # needs: fd (fdfind on Debian)
```

⚠ FOOTGUN
Never `for f in $(rg -l PAT); do …`. Command substitution word-splits on whitespace and glob-expands, so filenames with spaces, tabs, newlines, `*`, or `?` detonate. Use `rg -0 -l PAT | xargs -0`, or `while IFS= read -r -d '' `.

10. Piping `rg` Into Other Tools

Sink	Idiom
Pager	`rg -n --color=always 'PAT' | less -R`
Line count	`rg -c 'PAT' | awk -F: '{s+=$2} END{print s}'` (sum) · `rg -l 'PAT' | wc -l` (files)
Unique extract	`rg -oN 'PAT' | sort -u`

Frequency	`rg -oN 'PAT' | sort | uniq -c | sort -rn`
Field slice	`rg --vimgrep 'PAT' | cut -d: -f1-3`
JSON	`rg --json 'PAT' | jq -r '…'` (§14) needs jq
Interactive	`rg -n 'PAT' | fzf` needs fzf
Parallel batch	`rg -l -0 'PAT' | parallel -0 CMD` (or `xargs -0 -P`) needs parallel

```
rg -n --color=always 'PATTERN' | less -R                # keep colors in the pager
rg -oN 'https?://[^[:space:]"'\''<>]+' | sort -u         # dedup extracted URLs
rg -l -0 'PATTERN' | xargs -0 sed -n '1,120p'           # peek head of each hit file
rg -l -0 'PATTERN' | xargs -0 code                      # open all hits in VS Code
rg -n 'PATTERN' | fzf                                   # fuzzy-pick a match line
# annotate each matched file with its type
rg -l -0 'PATTERN' | xargs -0 -I{} sh -c 'printf "%s\t" "$1"; file -b "$1"' sh {}
```

✓ **FIELD TIP**
Keep NUL end-to-end: `-0` / `--null` on the rg side **and** `-0` (or `-print0`) on every downstream `xargs` / `parallel` / `sort -z`. A single non-NUL hop re-opens the whitespace-splitting hole you closed.

11. Git-Aware Workflows

rg does not read git history — it searches the working tree. Pair it with git's plumbing (always `-z` / NUL) to scope tracked, changed, or staged files. For **historical** content, materialize it first (`git show REV:path`).

```
# Tracked files only (skips untracked build junk without needing ignore rules)
git ls-files -z | xargs -0 rg -n 'PATTERN'

# Files this branch changed vs its merge-base with main
git diff --name-only -z main...HEAD | xargs -0 rg -n 'PATTERN'

# Staged content only — the natural pre-commit scope
git diff --cached --name-only -z | xargs -0 rg -n 'SECRET|TOKEN|PASSWORD'

# Commit messages (metadata, not tree contents)
git log --all --oneline --decorate | rg 'security|auth|token|secret'

# A file as it was at a past revision
git show v1.4.0:config/app.yml | rg -n 'debug|verbose'
```

⚠ **FOOTGUN**
`rg -n 'PATTERN' $(git ls-files)` is a trap: the command substitution word-splits on whitespace, so any tracked path containing a space or newline breaks (and huge repos blow the `ARG_MAX` limit). Always `git ls-files -z | xargs -0 rg`.

NOTE
`git grep` vs `rg`. `git grep` natively searches the index and arbitrary revisions (`git grep PAT REV`) and obeys `.gitattributes`. Prefer it for **history**. Prefer rg for the working tree: file-type logic, PCRE2, multiline, `--json`, and searching untracked/ignored files on demand.

12. Security & Audit Recipes

⚠ **DANGER**
Everything below finds candidates. A hit means "a human should read this line," not "this is a secret / a vulnerability / malicious." Expect false positives in any real tree. These are defensive triage aids for code you are authorized to review — never a substitute for a scanner, a secret manager, or a human reading the diff.

12.1. Credentials, keys, and tokens

```
# generic hardcoded secret assignment (skip minified & lockfiles)
rg -nP -g '!*.min.*' -g '!*.lock' \
  '(?i)\b(pass(wd|word)?|secret|api[_-]?key|token|access[_-]?key)\b\s*[:=]\s*["'\''][^"'\'']{6,}'

# private key blocks (multiline) and their one-line headers
# -e is REQUIRED: the pattern starts with '-' and would be parsed as a flag
rg -Un -e '-----BEGIN [A-Z ]*PRIVATE KEY-----' .

# cloud key ids + JWT-looking tokens
rg -nP '\b(AKIA|ASIA|AGPA|AIDA)[0-9A-Z]{16}\b'
rg -nP '\beyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}'
```

```
# Terraform / IaC inline secrets                                    # needs: -t tf
rg -nP -t tf '(?i)(password|secret|access_key|private_key|token)\s*=\s*"[^"]{6,}"'
```

12.2. Execution, download-exec, obfuscation

```
# curl|wget piped straight into a shell
rg -nP '\b(curl|wget)\b.{0,40}\|\s*(sudo\s+)?(ba)?sh\b'

# base64 decode + eval/exec on the same line (classic dropper shape)
rg -nP '(eval|exec)\s*\(\s*(atob|base64|Buffer\.from)'
rg -n  '\bbase64\s+(-d|-D|--decode)\b'          # macOS -D, GNU -d

# hex/unicode-escape obfuscation runs
rg -nP '(\\x[0-9a-fA-F]{2}){6,}'
```

12.3. Persistence & platform config

```
# shell startup persistence (search dotfiles explicitly, skip .git)
rg -n --hidden -g '.bashrc' -g '.zshrc' -g '.profile' -g '.bash_profile' -g '!.git' \
  '(curl|wget|base64|eval|/tmp/|nc\s|ncat|/dev/tcp)'

# cron-style scheduling in tracked configs
rg -n --hidden -g '!.git' '(@reboot|\*/\d+\s+\*|/etc/cron)'

# macOS LaunchAgents / LaunchDaemons                                # macOS-specific
rg -Un -g '*.plist' '<key>RunAtLoad</key>|<key>ProgramArguments</key>|<key>StartInterval</key>'

# npm lifecycle hooks that run on install
rg -n -g 'package.json' '"(pre|post)?install"\s*:|"scripts"\s*:'
```

12.4. Language-level dangerous sinks

```
rg -nP 'subprocess\.(run|call|Popen|check_output)\([^)]*shell\s*=\s*True'   # Python
rg -nP "require\(['\"]child_process['\"]\)|\b(exec|execSync|spawn)\s*\("     # Node
rg -nP '\b(pickle\.loads|yaml\.load\s*\(|marshal\.loads|readObject|unserialize)\b'  # deserialization
rg -nP '(?i)\b(select|insert|update|delete)\b.*(\+\s*["'\'']|["'\'']\s*\+|%s|\$\{)'  # SQL concat
```

12.5. Containers, Kubernetes, filesystem posture

```
rg -n -t docker '(?i)(ADD\s+https?://|curl.*\|\s*sh|--no-check-certificate|:latest\b|^USER\s+root)'
rg -Un -g '*.yaml' -g '*.yml' 'kind:\s*Secret|stringData:|^\s*privileged:\s*true'   # k8s (no -t k8s type)
rg -n 'chmod\s+(-R\s+)?(0?777|a\+w|o\+w)\b'          # world-writable candidates
rg -n '\bsetuid\b|\bu\+s\b|chmod\s+[0-6]?4[0-7]{3}\b' # SUID mentions
```

> **NOTE**
> Scope every audit: add `-g '!*.min.js' -g '!vendor/**' -g '!node_modules/**'` and consider `--no-ignore -g '!.git'` when you specifically want to see gitignored artifacts. Pipe to `--json` (§14) when the output feeds a report or ticket rather than your eyes.

13. Replacement & Extraction

> **⚠ DANGER**
> **`rg` never edits files.** `-r` / `--replace` rewrites matches **on stdout only**. It is a preview and an extractor, not an in-place editor. To apply changes use `sd`, `perl -pi`, `sed -i`, or your editor's project-wide replace — after you have reviewed the preview.

13.1. Extraction with `-o` (+ optional `-r`)

```
rg -oIN 'https?://[^[:space:]"'\''<>]+' | sort -u          # unique URLs
rg -oINP 'https?://\K[^/[:space:]"'\''<>]+' | sort -u      # just hostnames (\K, PCRE2)
rg -oINP '^\s*(?:from|import)\s+\K[\w.]+' -t py | sort -u  # unique Python imports
rg -oIN -r '$1' '^\s*export\s+([A-Z_][A-Z0-9_]*)=' -t sh | sort -u   # env var keys
rg -oINP '(?<=image:\s)\S+' -g '*.y*ml' | sort -u          # container image names
```

`-o` prints each match on its own line; `-I` (`--no-filename`) drops the path prefix so `sort -u` sees only values (essential across a multi-file tree); `-N` drops line numbers; `-r '$1'` isolates a capture group. Add `tr 'A-Z' 'a-z'` to normalize case before dedup.

13.2. Migration previews (review before applying)

```
# rename an API call — preview only
rg -n -r 'newFetch($1)' '\boldFetch\(([^)]*)\)'

# rewrite an import path prefix
rg -n -r 'import $1 from "@app/$2"' 'import (\w+) from "src/(\w+)"' -t ts

# migrate a config key name (preview the substitution in context)
rg -n -r '$1timeout_ms:' '^(\s*)connect_timeout:' -g '*.yaml'
```

13.3. Safe migration workflow

1. **Preview with `rg -n -r 'REPL' 'PAT'` — read every changed line.**
2. **Scope** with `-t`, `-g`, or a tracked-file list so blast radius is known.
3. **Apply** with a real editor: `rg -l -0 'PAT' | xargs -0 sd 'PAT' 'REPL'` needs sd — or `perl -pi -e` / `sed -i`.
4. **Re-scan** for stragglers and for accidental over-matches (`rg -n 'REPL'`).
5. **Diff & test.** `git diff` then run the suite. rg's preview is not a guarantee the apply tool used identical regex semantics.

> ⚠ **FOOTGUN**
> rg (default engine) and your apply tool may not share regex dialects. `sd` uses the same Rust `regex` crate (safe match), but `sed` is BRE/ERE and `perl` is PCRE — capture syntax and metacharacter rules differ. Re-verify the apply step's own regex; don't assume the rg pattern transfers verbatim.

14. JSON Mode & Machine-Readable Output

`--json` emits **JSON Lines** — one self-describing event object per line. Parse it; never scrape human output. A filename can legally contain `:`, a space, or a newline, which makes `file:line:text` ambiguous — JSON is not.

14.1. Event stream

`type`	Key fields under `.data`
`begin`	`path.text` — fires once per file that produces output
`match`	`path`, `lines.text`, `line_number`, `absolute_offset`, `submatches[]`
`context`	same shape as `match` (for `-A/-B/-C` lines); `submatches` empty
`end`	`path`, `binary_offset`, per-file `stats`
`summary`	`elapsed_total`, aggregate `stats` — the final event

Field	Meaning
`.data.path.text`	file path (or `.path.bytes` = base64 if not UTF-8)
`.data.lines.text`	the whole matching line incl. trailing `\n`
`.data.line_number`	1-based line number
`.data.submatches[].match.text`	each matched substring on the line
`.data.submatches[].start/.end`	0-based **byte** offsets within the line
`.data.stats.matches`	total matches (summary: whole run; end: this file)
`.data.stats.matched_lines`	count of lines that matched

> ⚠ **FOOTGUN**
> When matched content is not valid UTF-8, rg emits `{"bytes":"<base64>"}` instead of `{"text":"…"}` for that field. Robust parsers must handle both keys. Column is a **byte** offset; `start+1` gives the 1-based byte column, not a codepoint column.

14.2. jq recipes (verified against jq 1.8.2) needs jq

```
# path:line:match  (one row per submatch; multi-match lines repeat)
rg --json 'PAT' | jq -r 'select(.type=="match")
  | "\(.data.path.text):\(.data.line_number):\(.data.submatches[].match.text)"'

# list unique files with a match  (begin fires once per matching file)
rg --json 'PAT' | jq -r 'select(.type=="begin") | .data.path.text'

# extract just the matched text
rg --json 'PAT' | jq -r 'select(.type=="match") | .data.submatches[].match.text'

# matching-LINE count by file
rg --json 'PAT' | jq -r 'select(.type=="match") | .data.path.text' | sort | uniq -c | sort -rn

# total-MATCH count by file (pure jq, slurped)
rg --json 'PAT' | jq -rs 'map(select(.type=="match"))
  | group_by(.data.path.text)[]
  | "\([.[].data.submatches|length]|add)\t\(.[0].data.path.text)"'

# TSV audit report: path, line, 1-based col, text
rg --json 'PAT' | jq -r 'select(.type=="match")
  | [.data.path.text, .data.line_number, (.data.submatches[0].start+1),
     .data.submatches[0].match.text] | @tsv'

# JSON array artifact for a ticket / SARIF-ish feed
rg --json 'PAT' | jq -s 'map(select(.type=="match")
  | {file:.data.path.text, line:.data.line_number,
     text:(.data.lines.text|rtrimstr("\n"))})'

# run stats only
rg --json 'PAT' | jq -c 'select(.type=="summary") | .data.stats
  | {matches, matched_lines, searches, bytes_searched}'
```

✓ FIELD TIP
`jq -r` (raw) strips JSON quotes for shell consumption; `jq -s` (slurp) reads the whole stream into one array so you can `group_by` / `map`. Combine with `@tsv` / `@csv` for spreadsheets, or `@json` to re-emit filtered events.

15. Scripting With `rg`

15.1. Exit codes (verified)

Code	Meaning
`0`	at least one match found
`1`	no match (a **normal** result, not an error)
`2`	an actual error — bad regex, unreadable path, bad flag

⚠ FOOTGUN
Under `bash set -e`, a "no match" (exit 1) **aborts your script**. Verified: `bash -c 'set -e; rg -q zzz f; echo after'` never prints `after`. Guard every bare rg call: wrap it in `if`, or append `|| true` when you only want side effects. (`zsh` differs again — don't rely on cross-shell `set -e` parity.)

15.2. Quiet checks & guards

```
rg -q 'PAT' PATH            # no output; exits 0/1; stops at first match (fast)

if rg -q 'FORBIDDEN'; then echo "found"; exit 1; fi     # fail if present
rg -q 'REQUIRED' || { echo "missing marker" >&2; exit 1; }   # fail if absent

set +e; hits=$(rg -n 'PAT' src/); set -e   # collect without aborting on "no match"
```

15.3. NUL-safe filenames & arrays

```
# iterate matched files safely (spaces / newlines / globs in names)
while IFS= read -r -d '' file; do
  process "$file"
done < <(rg -l -0 'PAT')

# build an rg invocation as an array — never a string, never $(...)
args=(rg -n --color=never)
[[ -n "${TYPE:-}" ]]   && args+=(-t "$TYPE")
[[ "${HIDDEN:-}" == 1 ]] && args+=(--hidden -g '!.git')
"${args[@]}" -e "$PATTERN" -- "$SEARCH_PATH"
```

⚠ FOOTGUN
Use `-e PATTERN` (or a `--` terminator before paths) whenever a pattern or path can start with `-`; otherwise rg parses it as a flag. `rg -e '-v'` searches for the text `-v`; `rg '-v'` inverts the (missing) match.

15.4. Pattern files, allow/deny lists

```
rg -nf denylist.txt src/          # OR of every line in the file (one regex per line)
rg -n -f - src/ <<'PATS'          # inline pattern list via stdin
AKIA[0-9A-Z]{16}
-----BEGIN [A-Z ]*PRIVATE KEY-----
PATS
```

15.5. CI checks

```
#!/usr/bin/env bash
set -euo pipefail

# 1) denylist guard — fail the build if any forbidden shape appears
if rg -nP -g '!*.min.*' -g '!*.lock' \
     'AKIA[0-9A-Z]{16}|-----BEGIN [A-Z ]*PRIVATE KEY-----'; then
  echo "::error:: secret-shaped string in tree" >&2
  exit 1
fi

# 2) changed-files-only secret audit (PR diff vs main)
git diff --name-only -z origin/main...HEAD \
  | xargs -0 -r rg -nP 'SECRET|TOKEN|PASSWORD' \
  && { echo "::warning:: review changed-file candidates"; }

# 3) block a removed API from creeping back
if rg -q --no-heading 'deprecatedCall\(' src/; then
  echo "deprecatedCall() was removed in v2 — update callers" >&2
  exit 1
fi
```

⚠ **FOOTGUN**
Empty-input behavior **differs**: **GNU** `xargs` runs the command **once with no args** on empty input — so `xargs -0 rg PAT` with an empty diff would search all of `.`. Fix with `-r` / `--no-run-if-empty`. **BSD/macOS** `xargs` already skips on empty input (verified) and accepts `-r` as a harmless no-op. Always write `xargs -0 -r` — portable and safe on both.

16. Performance & Scale

rg is fast because it **prunes before it reads** (gitignore/hidden), searches files in parallel, scans literals with SIMD, and runs a linear-time DFA. It is not magic: it is still bound by I/O, by how many files survive pruning, and by whatever you force it to do (multiline buffering, PCRE2 backtracking, `--sort` serialization). Speed is mostly about **shrinking the candidate set**.

16.1. Levers, cheapest first

Lever	Effect
Type / glob narrowing (`-t` , `-g`)	Fewer files opened. The single biggest win.
Keep default ignores on	`.gitignore` pruning skips `node_modules` , `target` , etc. for free
`--max-filesize 2M`	Skip giant minified/vendored/data blobs
`-d/--max-depth N`	Stop deep recursion in sprawling trees
Pre-filter file list	`git ls-files` / `fd` / `find` → `xargs -0 rg` when the **set** is the cost
Avoid `--no-ignore` / `-uu`	Re-adds vendored + `.git` noise; opens far more files
Avoid `--sort` in hot paths	Sorting forces **single-threaded** traversal
Avoid `-U` on huge files	Multiline buffers larger regions; higher RAM + time
Avoid `-P` unless needed	PCRE2 can backtrack; default engine is linear
`--one-file-system`	Don't wander onto slow network/other mounts

NOTE
There is **no** built-in `code / all` aggregate type. Make one once:
`rg --type-add 'code:*.{rs,go,py,js,ts,java,rb,c,cc,cpp,h,hpp}' -t code 'PAT'` (or persist it in `.ripgreprc` , §17).
Then `-t code` narrows the whole search cheaply.

16.2. Profiles

Fast repo search

Narrow by type; trust default ignores.

```
rg -t py -t js 'PATTERN'
```

Deep repo audit

See ignored + hidden, minus the git store.

```
rg -uu -g '!.git' -g '!node_modules/**' 'PATTERN'
```

Huge monorepo, targeted

Let git enumerate; scope to a subtree/type.

```
git ls-files -z 'services/api/**' \
  | xargs -0 rg -t go 'PATTERN'
```

Hidden-file audit

Dotfiles yes, object store no.

```
rg --hidden -g '!.git' 'PATTERN'
```

Log directory scan

Cap runaway lines; a little context.

```
rg -t log -C1 --max-columns 300 \
   --max-columns-preview 'ERROR|panic' /var/log
```

Generated-noise avoidance

One reusable exclude set.

```
rg -g '!
{dist,build,target,vendor,node_modules,.venv,__pycache__,cover
**' 'PATTERN'
```

17. Configuration

rg reads **exactly one** config file: the path named by `RIPGREP_CONFIG_PATH` . There is **no** implicit `~/.ripgreprc` . Each line is **one argument** — not a shell line. Do not quote globs, do not use shell expansion, one flag per line, `#` for comments.

```
export RIPGREP_CONFIG_PATH="$HOME/.config/ripgrep/ripgreprc"
```

Safe default rc

```
# ~/.config/ripgrep/ripgreprc
--smart-case
--max-columns=200
--max-columns-preview
--colors=path:style:bold
--colors=match:fg:red
# a personal aggregate type
--type-add=code:*.{rs,go,py,js,ts,java,rb}
```

Dedicated audit rc (switch via env)

```
# ~/.config/ripgrep/audit.rc
--hidden
--no-ignore-vcs
--glob=!.git
--glob=!node_modules
--line-number
```

```
alias rga='RIPGREP_CONFIG_PATH=~/.config/ripgrep/
audit.rc rg'
```

⚠ **FOOTGUN**
Quoting in `.ripgreprc` is a classic break. `--glob='!.git'` (verified) makes the quotes **part of the glob** and silently stops excluding `.git`. Write `--glob=!.git`. Same for `--colors`, `--type-add`: no quotes, no spaces around `=` beyond what the flag expects.

⚠ **FOOTGUN**
A global rc leaks into **everything**, including scripts and other tools that shell out to rg. `--smart-case`, `--hidden`, or `--no-ignore` set globally will silently change results and exit codes in a CI job. Keep the default rc minimal; put aggressive flags in a **named** audit rc you opt into. In CI, either `unset RIPGREP_CONFIG_PATH` or pin it explicitly.

Shell-level ergonomics (these live in `.zshrc` / `.bashrc`, not the rc file):

```
rgx() { rg -nP -g '!.git' -g '!node_modules/**' "$@"; }   # audit-flavored rg
rgf() { rg --files | rg -i "$1"; }                        # fuzzy-ish filename find
alias rgt='rg --type-list | rg'                            # "what type covers X?"
```

18. Troubleshooting

18.1. Symptom → cause → fix

"No matches" but you expect some	Diagnose / fix
File is git-ignored	`rg --no-ignore-vcs PAT` or `rg -uu PAT`; confirm via `rg --files | rg name`
File is hidden (dotfile)	`rg --hidden PAT` (add `-g '!.git'`)
File is binary (has NUL)	`rg -a PAT` / `--text`; or `rg -uuu`
Pattern needs multiple lines	add `-U` (and `-P` if you used lookaround)
Used lookaround/backref	add `-P`; default engine rejects them
Shell ate the pattern	single-quote it: `rg 'a|b'` not `rg a|b` (pipe = shell pipe!)
Shell expanded the glob	quote it: `rg -g '*.py'`; zsh errors on unmatched globs
Regex meta not escaped	`\.` for a dot, or `-F` for a literal search
Pattern looked like a path/flag	`rg -e '-x'` or `rg -- 'PAT' PATH`
Searching stdin, lost filenames	give file args instead of `cat file | rg`
`xargs` split a filename	use `rg -l0 … | xargs -0`; never `$(...)`
No color in a pager	`rg --color=always PAT | less -R`
Too much output	`-l`, `-c`, `--max-columns`, `-m N`, or narrow with `-t` / `-g`
Slow	narrow types/globs; drop `--no-ignore` / `-uu`; `--max-filesize`; see §16

18.2. Diagnosis commands

```
rg --version                # build + whether +pcre2 is present
rg --pcre2-version          # errors if this build lacks PCRE2
command -v rg               # is it even on PATH / which one
rg --files                  # exactly which files a search would consider
rg --files --hidden         # …including dotfiles
rg --files --no-ignore      # …including git-ignored files
rg --type-list              # all type names + their globs
rg --files -g 'GLOB'        # does my glob select what I think?
rg --debug 'PAT' 2>&1 | rg -i 'ignore|glob|whitelist'   # why a file was skipped
```

NOTE
macOS vs Linux. Coreutils differ: `base64 -D` (macOS) vs `-d` (GNU); `sed -i ''` (macOS needs the empty suffix arg) vs `sed -i` (GNU); GNU `xargs` runs once on empty input, BSD skips (§15). rg itself behaves the same; the **tools around it** in your pipeline are where portability bites.

NOTE
zsh vs bash quoting. In zsh an unmatched glob like `rg -g *.py` is a hard error (`no matches found`); in bash it passes through. `!` triggers history expansion in interactive bash (`"…!…"`) but not in zsh. Both: single-quote patterns and globs and the differences disappear. In zsh you can also `setopt NO_NOMATCH` to soften glob errors, but quoting is cleaner.

19. Expert Combos

The payoff section: real problems, the exact shape that solves them, why it is built that way, and the trap. All commands assume the verified rg 15.1.0.

1 · Changed-files-only security audit

```
git diff --name-only -z --diff-filter=d origin/main...HEAD \
  | xargs -0 -r rg -nP -g '!*.min.*' \
      'AKIA[0-9A-Z]{16}|-----BEGIN [A-Z ]*PRIVATE KEY-----|(?i)(secret|token|password)\s*[:=]\s*\S{6,}'
```

WHY Scopes to just what the PR touched — fast and low-noise. NUL end-to-end; PCRE2 for the alternation of shapes; `-r` no-ops an empty diff.
FOOT `--diff-filter=d` drops **deleted** paths so rg doesn't error (exit 2) on a missing file. `...` = changes since the merge-base, not the raw two-endpoint diff.

2 · Extract URLs from tracked files only

```
git ls-files -z | xargs -0 rg -oIN 'https?://[^[:space:]"'\''<>]+' | sort -u
```

WHY Tracked-only skips build/vendor without ignore gymnastics; `-I` drops path prefixes so `sort -u` dedups on the URL itself.
FOOT Tracked **binary** assets are skipped by default; URLs inside minified JS still land unless you add `-g '!*.min.*'`.

3 · JSON audit report with file / line / match

```
rg --json -nP 'PATTERN' src/ \
  | jq -r 'select(.type=="match")
      | [.data.path.text, .data.line_number, .data.submatches[0].match.text] | @tsv' \
  > findings.tsv
```

WHY JSON is unambiguous even when paths contain `:`; `@tsv` is spreadsheet- and `cut`-ready. needs jq
FOOT Only the **first** submatch per line is emitted here; drop `[0]` and iterate `.submatches[]` to capture every hit. Non-UTF-8 → `.bytes` (base64), not `.text`.

4 · PCRE2 secret-candidate search with globs

```
rg -nP -g '!*.min.js' -g '!*.lock' -g '!vendor/**' \
   '(?i)(api[_-]?key|secret|token)\s*[:=]\s*["'\''][^"'\'']{8,}["'\'']'
```

WHY PCRE2 handles the case-insensitive class + quoted-value capture cleanly; globs strip the highest-FP directories up front.
FOOT Candidates only (§12). Misses base64-embedded or split-string secrets, and any value under 8 chars.

5 · Multiline private-key candidate search

```
rg -Un -g '!.git' -e '-----BEGIN [A-Z ]*PRIVATE KEY-----[\s\S]*?-----END [A-Z ]*PRIVATE KEY-----' .
```

WHY `-U` spans the block; `[\s\S]*?` matches any char incl. newlines **lazily** — no `-P` needed, so it stays linear-time. `-e` is mandatory because the pattern starts with `-`.
FOOT Finds keys embedded in **text**; won't see DER/binary keystores or PEM split across non-adjacent regions.

6 · Shell startup / persistence audit

```
rg -n --hidden -g '!.git' \
   -g '.bashrc' -g '.zshrc' -g '.profile' -g '.bash_profile' -g '.zprofile' \
   '(curl|wget|base64|eval|/dev/tcp|/tmp/|nc\s)' ~
```

WHY Explicit dotfile globs + `--hidden` reach the files that run at login; `~` targets the home tree.
FOOT Version managers (nvm, rbenv, pyenv) legitimately match `eval`. Triage, don't alarm.

7 · Package lifecycle-script audit

```
rg -Un -g 'package.json' '"scripts"\s*:\s*\{[\s\S]*?\}' \
  | rg -nP '(pre|post)?install|\b(curl|wget|node\s+-e|rimraf)\b'
```

WHY First pass isolates the `scripts` object (multiline); second pass flags install hooks and risky commands inside it.
FOOT Regex over JSON is approximate — nested braces defeat `[^}]`. For anything load-bearing, parse with `jq '.scripts'`. needs jq

8 · Docker / Kubernetes config audit

```
rg -n -t docker '(?i)(^FROM .*:latest|^USER\s+root|ADD\s+https?://|curl.*\|\s*sh)'
rg -Un -g '*.y*ml' 'kind:\s*Secret|^\s*privileged:\s*true|hostNetwork:\s*true'
```

WHY `-t docker` uses the built-in `*Dockerfile*` glob; there is no `k8s` type, so manifests go through a yaml glob + multiline.
FOOT YAML truth needs a parser (`yq`). `:latest` and `USER root` are smells, not verdicts.

9 · Deprecated-API migration preview

```
rg -n -r 'newClient($1)' '\boldClient\(([^)]*)\)' -t go
```

WHY `-r` previews the capture-group rewrite in context **without touching files**; `-t go` scopes the blast radius.
FOOT Preview only — apply with `sd` / `gofmt` -aware tooling. Calls that wrap across lines are missed without `-U`.

10 · Monorepo targeted search profile

```
git ls-files -z apps/web ':!*.snap' | xargs -0 rg -t ts -n 'PATTERN'
```

WHY `apps/web` is a **leading-directory** git pathspec (everything under it); `:!*.snap` excludes at the VCS layer — cheap. rg then only opens TypeScript. For real `**` globbing, prefix with `:(glob)`.
FOOT These are **git** pathspecs, not rg globs — quote them; here `*` matches `/`, unlike a shell/`-g` glob. Semantics differ.

11 · Hidden-file audit excluding .git

```
rg -n --hidden -g '!.git' -g '!.terraform/**' 'PATTERN'
```

WHY `--hidden` alone adds dotfiles but **keeps** `.git`; the excludes drop the object store and provider caches.
FOOT `-uu` would also re-add **ignored** files and `.git` (verified §6). Use plain `--hidden` for hidden-only.

12 · Log scan with context + column cap

```
rg -t log -n -C2 --max-columns 400 --max-columns-preview \
   '(?i)\b(error|fatal|panic|segfault|oom)\b' /var/log
```

WHY Context lines give causality; the column cap stops a single 1 MB JSON log line from flooding the terminal (preview still shows a head).
FOOT `/var/log` often needs `sudo`; rotated `*.gz` need `-z` / `--search-zip` (§6).

13 · Open results in an editor safely

```
rg -l -0 'PATTERN' | xargs -0 "${EDITOR:-vim}"        # or: xargs -0 code -g
```

WHY NUL-delimited list survives odd filenames; `${EDITOR:-vim}` respects the environment.
FOOT This opens **every** hit at once — pre-count with `rg -l 'PATTERN' | wc -l` before unleashing it on a big match set.

14 · fzf interactive match picker

```
rg --line-number --no-heading --color=always 'PATTERN' \
  | fzf --ansi --delimiter : --nth 3.. \
        --preview 'sed -n "{2}p" {1}' --preview-window up:3
```

WHY rg feeds `file:line:text`; fzf searches the text field (`--nth 3..`) and previews the exact line via `sed`. needs fzf · sed
FOOT Filenames containing `:` desync the `--delimiter`; rare in code trees but real. Empty query = no preview target.

15 · Null-safe parallel batch processor

```
rg -l -0 'PATTERN' | xargs -0 -P4 -I{} sh -c 'process "$1"' _ {}
```

WHY `-0` end-to-end; `-P4` runs four workers; `_ {}` passes the filename as `$1` (not `$0`) so quoting is safe.
FOOT `-I{}` forces one process **per file** (no batching) — slow for trivial ops. Drop `-I{}` and let `sh -c 'for f; do …' _ "$@"` batch.
needs GNU xargs for -P on old macOS

16 · CI guard with allow-list

```
#!/usr/bin/env bash
set -euo pipefail
findings=$(rg -nP -f denylist.txt -g '!*.min.*' src/ | rg -v -f allowlist.txt || true)
[[ -z "$findings" ]] || { printf '%s\n' "$findings" >&2; exit 1; }
```

WHY `denylist.txt` raises candidates; `rg -v -f allowlist.txt` removes accepted lines; `|| true` stops `set -e` aborting on a clean (no-match) run.
FOOT Content-based allow-listing is brittle. Prefer inline `# pragma: allowlist` markers or a purpose-built scanner once this grows.

17 · Custom type + project ignore file audit

```
rg -n --type-add 'iac:*.{tf,tfvars,yaml,yml}' -t iac \
   --ignore-file .auditignore \
   '(?i)(password|secret|access_key)\s*='
```

WHY One-shot aggregate type for IaC plus an extra, checked-in `.auditignore` to prune known-safe fixtures — repeatable without touching global config.
FOOT `--ignore-file` **adds** excludes on top of the defaults; it doesn't replace them. Keep the ignore file in the repo so CI and humans agree.

18 · Compare match counts across a branch

```
git worktree add -q --detach /tmp/wt-base main
cmp() { rg -c 'PATTERN' "$1" | sed "s|^$1/||" | sort; }
diff <(cmp .) <(cmp /tmp/wt-base)
git worktree remove --force /tmp/wt-base
```

WHY A **detached** worktree checks out `main`'s commit elsewhere so your tree is untouched — `--detach` sidesteps "branch already checked out" when you're on main; `-c` gives per-file counts; `sed` strips the path prefix so `diff` compares like-for-like.
FOOT Needs a clean state + disk for the worktree. `-c` counts matching **lines**; swap to `--count-matches` for raw totals.

20. Appendix — Quick Reference

20.1. A · Flags by purpose

Matching	
`-i -S -s`	ignore / smart / force case
`-w -x`	word / whole-line anchor
`-F`	literal (fixed strings)
`-v`	invert (non-matching lines)
`-U`	multiline
`-P`	PCRE2 engine
`-e / -f`	pattern arg / pattern file
`-m N`	stop after N matches per file

Scope	
`-t / -T`	include / exclude type
`-g / --iglob`	glob (case-insens.)
`--hidden`	include dotfiles
`--no-ignore*`	relax ignore rules
`-u/-uu/-uuu`	no-ignore / +hidden / +binary
`-d / --max-filesize`	depth / size cap
`-L / --one-file-system`	symlinks / mount bound
`-a / -z`	text / search-zip

Output	
`-n/-N -H/-I`	line-no / filename toggles
`--column --vimgrep`	byte col / quickfix
`-o -r`	only-match / replace-preview
`-A/-B/-C -C`	context lines
`--heading --trim`	group / strip indent
`--json`	event stream
`-0 --null-data`	NUL out / NUL-in records
`--sort/--sortr`	deterministic (serial)

Introspect	
`--files`	list searchable files
`--type-list`	all type definitions
`--debug`	why files were skipped
`--stats`	match/among-files counts
`--version`	build + pcre2 flag
`--pcre2-version`	PCRE2 lib version
`-q`	quiet: exit status only
`-c / --count-matches`	lines / total matches

20.2. B · Pattern & glob snippets

Pattern	Matches
`\bword\b`	whole word
`^\s*import`	line start + WS
`foo|bar`	alternation
`(?:ab)+`	non-cap repeat
`[[:xdigit:]]{7,40}`	hex run (POSIX)
`\p{Lu}`	any uppercase (Unicode)
`(?i:AbC)`	scoped case-insens.
`(?<=k=)\w+`	lookbehind (`-P`)

Glob	Selects
`-g '*.py'`	by extension
`-g '!**/test/**'`	exclude a dir tree
`-g '*.{ts,tsx}'`	brace set
`-g 'src/**/*.go'`	path-scoped
`--iglob '*.md'`	case-insensitive
`-g '!*.min.*'`	drop minified
`-g '.*'` (+type-add)	dotfiles as a type
`-g '!{dist,vendor}/**'`	common junk

20.3. C · File types & ignore hierarchy

Type command	
`rg -t py 'x'`	only Python
`rg -T js 'x'`	everything but JS
`rg --type-list`	names + globs
`--type-add 'n:*.ext'`	define a type
`--type-clear n`	wipe + redefine
built-ins	`sh py js ts docker tf yaml toml log config …`

Ignore precedence (high→low)	
CLI `-g` globs	last glob wins
`--ignore-file`	added excludes
`.rgignore`	rg-specific
`.ignore`	tool-agnostic
`.gitignore` + parents	VCS
hidden / binary skip	defaults

20.4. D · Output modes · exit codes · pipeline safety

Profile	Command
Human	`rg -p 'x'`
Editor	`rg --vimgrep 'x'`
Script	`rg -N --no-heading --color=never 'x'`
NUL list	`rg -0 -l 'x' | xargs -0`
Machine	`rg --json 'x' | jq`
Report	`rg -n --sort path --trim --max-columns 200 'x'`

Exit	Means
`0`	match found
`1`	no match (not an error)
`2`	error (regex/path/flag)

Pipeline safety rules

1 · Single-quote patterns and globs. 2 · `-0` / `--null` on rg **and** `-0` on `xargs` / `sort` / `parallel`. 3 · Never `for f in $(rg -l …)`; use `xargs -0` or `while read -r -d ''`. 4 · `xargs -0 -r` (empty-input safe on GNU + BSD). 5 · `-e PATTERN` or `--` when a pattern/path starts with `-`. 6 · Give an explicit path in scripts — no path + non-TTY stdin makes rg read stdin. 7 · `--color=never` into files/parsers; `--color=always | less -R` for pagers.

20.5. E · PCRE2 & multiline quick reference

PCRE2 (`-P`)	
`(?=…)` `(?!…)`	look-ahead pos/neg
`(?<=…)` `(?<!…)`	look-behind (fixed-width)
`\K`	reset match start
`\1` `(?<n>…)\k<n>`	back-ref / named
`a*+` `(?>…)`	possessive / atomic
`\A` `\z`	buffer start / end
cost	can backtrack — anchor + be specific

Multiline (`-U`)	
`-U`	pattern may cross `\n`
`--multiline-dotall`	`.` also matches `\n`
`(?s)` `(?m)`	dotall / multi-anchor (inline)
`[\s\S]*?`	any char, lazy — no `-P`
`+ -P`	needed for `(?=…)`, `\z` under `-U`
cost	buffers larger regions; more RAM
truth	structured data → use a parser

20.6. F · Common footguns (one-line)

- `-L` **is** `--follow`**, not list-non-matching (that's** `--files-without-match`**, no short flag).**
- `-c` counts matching **lines**; `--count-matches` counts **matches**.
- Default is case-**sensitive**; `-S` opts into smart-case.
- `--hidden` / `-uu` re-add `.git/**` — pair with `-g '!.git'`.
- `-r` previews on stdout; rg **never** edits files in place.
- `--column` and JSON `start` are **byte** offsets, not codepoints.
- Lookaround/backrefs need `-P`; `-U` alone won't enable them.
- Quotes in `.ripgreprc` become literal — write `--glob=!.git`.
- bash `set -e` + `rg -q` no-match (exit 1) aborts the script — guard with `if` / `|| true`.
- There is no `code` / `k8s` built-in type — build one with `--type-add`.

20.7. G · Recommended command profiles

Daily	`rg -n 'PAT'` · `rg -n -t TYPE 'PAT'` · `rg -l 'PAT'`
Editor	`rg --vimgrep 'PAT'`
Extract	`rg -oIN 'PAT' | sort -u`
Tracked audit	`git ls-files -z | xargs -0 rg -nP 'PAT'`
Changed-only	`git diff --name-only -z --diff-filter=d main...HEAD | xargs -0 -r rg -nP 'PAT'`
Secret sweep	`rg -nP -g '!*.min.*' -g '!*.lock' 'CANDIDATE_PAT'` (triage by hand)
JSON report	`rg --json 'PAT' | jq -r 'select(.type=="match")|[.data.path.text,.data.line_number]|@tsv'`
CI guard	`if rg -q 'FORBIDDEN' src/; then exit 1; fi`
Deep audit	`rg -uu -g '!.git' -g '!node_modules/**' -nP 'PAT'`

Verified against ripgrep 15.1.0 (`+pcre2`, PCRE2 10.45, JIT) and jq 1.8.2 on macOS (Darwin arm64). Flag names, short/long pairings, exit codes, `--json` event shapes, `.ripgreprc` parsing, and every non-interactive command in the Combos and Security sections were executed on this environment while writing. Interactive examples (`fzf`, `code`, `less`) are shown in canonical form and marked with their tool dependency. Regex-based security patterns identify **candidates only**, never proof of compromise. Rebuild with `./build.sh`.